

THE UNIVERSITY OF
MELBOURNE

Final Report

The Art of Scientific Computing: Project Title: High Dimensional Kalman Filters

Author:
Steven Pavli

Subject Handbook code:
COMP-90072

Faculty of Science
The University of Melbourne

Original Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Contents

1 Introduction . . . 1
2 Background . . . 1
3 Derivations . . . 1
4 Dynamic Projections . . . 2
5 Dynamic PCA-KF . . . 3
6 Results . . . 4
7 Discussion . . . 5

Bibliography **5**

1 Introduction

A Kalman filter (KF) is a type of algorithm which aims to estimate a hidden state in a linear system, given a series of observations. In particular, it handles the case in which the state variable evolves over time, and does so in a mean-squared optimal manner. This project aims to implement such an algorithm in the case of high dimensional linear regression, via the use of principle components analysis (PCA). The combination of PCA and KF exists in the literature, [SOURCE], however, the papers largely deal with a single PCA fit and never update this representation. To extend this, we experiment with a new type of rotating covarinace structure, and test a way of keeping the filter in the correct space via a method we term *projection chaining*. We compare projection chaining to the usual static approach and discuss some benefits and limitations of the method.

2 Background

- Introduce Kalman Filters, mean-square optimality.
- Application examples.
- Cite the PCA-KF review paper.
- Give time complexity and motivate the use of PCA.

The idea of running the KF in reduced-dimension PCA space is a proven method, as described in a recent review by Li et al. (2026).

The theme of this project is investigating the use of the Kalman Filter on high-dimensional time series data. We can specify the model as follows:

$$\boldsymbol{\beta}_t = \boldsymbol{\beta}_{t-1} + \boldsymbol{\varepsilon}_t \tag{1}$$

$$y_t = \boldsymbol{\beta}_t^T \boldsymbol{x}_t + z_t \tag{2}$$

where $\boldsymbol{x}_t, \boldsymbol{\beta}_t \in \mathbb{R}^p$, $p \geq 1$ large, and $\boldsymbol{\varepsilon}_t \sim N(0, \boldsymbol{\Sigma}_t)$, $z_t \sim N(0, \tau_t^2)$. Note that z_t is independent of $\boldsymbol{x}_t$. Equations (1) and (2) are the state and observation process equations respectively. In this case, we take $(\boldsymbol{x}_t, y_t)_{1 \leq t \leq T}$ be observed, while $\boldsymbol{\beta}_t$, $\boldsymbol{\Sigma}_t$, τ_t are unobserved (latent).

The goal is to efficiently estimate the $(\boldsymbol{\beta}_t)$ process (the loadings) via a Kalman Filter. To this end, we will consider a dimensionality reduction approach by applying PCA to $\boldsymbol{x}_t$, and running the Kalman Filter in this space, then back-transform our loading estimates at the end.

We will make use of Python, and NumPy in particular for fast matrix operations.

3 Derivations

Notation:

- $\boldsymbol{\beta}_t$: the state variable

- $\boldsymbol{Q}_t = \boldsymbol{\Sigma}_t$ is the covariance of the state process noise.
- $\boldsymbol{P}_{t|t}$ is the covariance of our estimate of $\boldsymbol{\beta}_t$.
- $R_t = \tau_t^2$ is the observation noise.

Prediction:

$$\begin{aligned}\boldsymbol{\beta}_{t|t-1} &= \boldsymbol{\beta}_{t-1|t-1} \\ \boldsymbol{P}_{t|t-1} &= \boldsymbol{P}_{t-1|t-1} + \boldsymbol{\Sigma}_t\end{aligned}$$

Update:

$$\begin{aligned}\tilde{y}_t &= y_t - \boldsymbol{\beta}_{t|t-1}^T \boldsymbol{x}_t \\ S_t &= \boldsymbol{x}_t^T \boldsymbol{\Sigma}_t \boldsymbol{x}_t + \tau_t^2 \sim \mathrm{Cov}(\tilde{y}_t) \\ K_t &= \cdots\end{aligned}$$

3.1 Initialisation

TO ADD

3.2 PCA-KF

The idea is that if p is large, then we apply dimension reduction to $\boldsymbol{x}_t$. This assumes the data generating process doesn't change 'too much' in the training set.

Observe that if $UU^T \approx I_p$, then

$$\boldsymbol{y}_t \approx \boldsymbol{\beta}_t^T UU^T \boldsymbol{x}_t + \boldsymbol{z}_t$$

This is the case, for example, if U represent PCA loadings with sufficient proportion of variance retained.

4 Dynamic Projections

Suppose we wish to update our PCA loadings as we progress through the iterations. This is a reasonable feature since the loadings estimates may become stale if we keep them static after initialisation. However, one issue arises from this; if we change our projection, then the usual update equations do not adapt to this. For example, the update $P_{k+1} = P_k + Q_k$ assumes the representation given by the k-th projection, likewise for updating the state estimate.

The first problem arises if we have a state $\boldsymbol{\beta}$, and assume we have an estimate of it in projected space: $\hat{\boldsymbol{\beta}}^U \approx \boldsymbol{\beta} U$, for $U \in \mathbb{R}^{p \times q}$, $q < p$. Then, suppose we wish to find the estimate in a different projected space, say, $\hat{\boldsymbol{\beta}}^V \approx \boldsymbol{\beta} V$, for $V \in \mathbb{R}^{p \times q'}$, $q' < p$. Additionally, suppose U and V arise from PCA so that $U^T U = I_q$ and $V^T V = I_{q'}$. Since we never observe $\boldsymbol{\beta}$ (or indeed, any estimate in the unprojected space), then we need an 'optimal' mapping from $\hat{\boldsymbol{\beta}}^U$ to $\hat{\boldsymbol{\beta}}^V$. We formalise this by the following optimisation problem:

$$\min_{W \in \mathbb{R}^{q \times q'}} \|\boldsymbol{\beta}^U W - \boldsymbol{\beta}^V\|_2$$

This is precisely a least squares problem, and the solution is

$$\begin{aligned}W &= ((\boldsymbol{\beta}^U)^T \boldsymbol{\beta}^U)^{-1} (\boldsymbol{\beta}^U)^T \boldsymbol{\beta}^V \\ &= (U^T \boldsymbol{\beta}^T \boldsymbol{\beta} U)^{-1} U^T \boldsymbol{\beta}^T \boldsymbol{\beta} V \\ &= U^T V\end{aligned}$$

So the least-square optimal state representation in V space is $\hat{\boldsymbol{\beta}}^V = \hat{\boldsymbol{\beta}}^U U^T V$. One can intuitively think of this as a mapping chain from $\mathbb{R}^q \xrightarrow{(1)} \mathbb{R}^p \xrightarrow{(2)} \mathbb{R}^{q'}$, where $(1 = U^T)$ tries to optimally 'undo' PCA and reconstruct the native space state, then $(2 = V)$ maps to $\mathbb{R}^{q'}$ via PCA.

We note that every time we apply an approximate projection, we permanently lose information in the direction of the bases lost in the prior projection. As such, we anticipate that this approach should not be chained for an extended number of iterations if the gross shift in the PCA loadings is expected to be very large.

4.1 Rotating Covariance

We propose a modification to the usual structure by allowing the PCA-loadings of the $\boldsymbol{x}$ data to rotate over time, while keeping the component variances (eigenvalues) constant. We can do this by using a block-rotation matrix R with reasonably small angles, and then we create the new covariance matrix via,

$$\Sigma' = R\Sigma R^T$$

Observe that implicitly we are rotating the PCA loadings since $R\Sigma R^T = RQ\Lambda Q^T R^T = Q'\Lambda(Q')^T$, where $Q' = RQ$. To be more precise, R is constructed as $R = \text{diag}(R(\theta_1), \ldots, R(\theta_k))$ for $k = p/2$ if p is even, else $R = \text{diag}(1, R(\theta_1), \ldots, R(\theta_{\lfloor p/2 \rfloor}))$; see (https://math.stackexchange.com/users/29335/rschwieb) n.d. Hence, to rotate the data incrementally, we can start with some covariance matrix Σ_0, then define:

$$\Sigma_n = R^n \Sigma_0 (R^n)^T$$

There is a question of how to best optimise this for a large dataset. One can directly construct R^n for each n by replacing $R(\theta_j)$ with $R(n\theta_j)$ for each $j = 1, \ldots, k$, hence eliminating the need to compute the matrix power. From there, we can batch multiply to calculate the entire sequence $(\Sigma_1, \ldots, \Sigma_N)$ simulataneously, provided memory constraints are respected.

In context of the projection chaining apporach, perhaps it is viable if the accumulated subspace rotation across the dataset is small.

4.2 Evaluating Approximation Error

Due to information loss which results from the projection chaining, we can test how much information is preserved as compared to simply projecting once initially. This can give us a sense of how we should best project our data during the Kalman filter steps.

5 Dynamic PCA-KF

This applies the previous projection chaining apporach to the PCA-KF algorithm. The algorithm is given as follows:

1. **Predict:**
   (a) $\boldsymbol{P}_{t|t-1} = \boldsymbol{P}_{t-1|t-1} + \boldsymbol{Q}_{t-1|t-1}$
   (b) $\boldsymbol{\beta}_{t|t-1} = \boldsymbol{\beta}_{t-1|t-1}$
2. **Project:**
   (a) Acquire the new PCA loadings $\boldsymbol{U}_t$.
   (b) Acquire the pushforward map: $\boldsymbol{V}_t = \boldsymbol{U}_{t-1}^T \boldsymbol{U}_t$
   (c) $\boldsymbol{P}_{t|t-1} \leftarrow \boldsymbol{V}_t^T \boldsymbol{P}_{t|t-1} \boldsymbol{V}_t$
   (d) $\boldsymbol{Q}_{t|t-1} \leftarrow \boldsymbol{V}_t^T \boldsymbol{Q}_{t|t-1} \boldsymbol{V}_t$
   (e) $\boldsymbol{\beta}_{t|t-1} \leftarrow \boldsymbol{\beta}_{t|t-1} \boldsymbol{V}_t$
   (f) $\boldsymbol{z}_t \leftarrow \boldsymbol{z}_t \boldsymbol{V}_t$
3. **Update:**
   (a) $\delta_t = y_t - \boldsymbol{\beta}_{t|t-1}^T \boldsymbol{z}_t$
   (b) $S_t = \boldsymbol{z}_t^T \boldsymbol{P}_{t|t-1} \boldsymbol{z}_t + R$
   (c) $\boldsymbol{K}_t = S_t^{-1} \boldsymbol{P}_{t|t-1} \boldsymbol{z}_t$
   (d) $\boldsymbol{\beta}_{t|t} \leftarrow \boldsymbol{\beta}_{t|t-1} + \delta_t \boldsymbol{K}_t$
   (e) $\boldsymbol{P}_{t|t} \leftarrow (I - \boldsymbol{K}_t \boldsymbol{z}_t^T) \boldsymbol{P}_{t|t-1} (I - \boldsymbol{K}_t \boldsymbol{z}_t^T)^T + R \boldsymbol{K}_t \boldsymbol{K}_t^T$

We note that the Predict and Project steps can be swapped to no effect.

6 Results

Which plots/tables to provide?

- Plot of average proportion of time that the chained proportion is greater than or equal to the naive proportion: line plot with error bars, against proportion of variance.
- Boxplot of Mahalanobis distances based on chaining frequency (inc. no chaining).
- Mahalanobis distance against: proportion of variance retained (fixed seed).
- Speed of the algorithm based on: (a) chaining frequency, (b) proportion of variance retained.
- (OPTIONAL) Runtime comparison: CPU vs CUDA.

6.1 Dynamic Projection Tests

First we present a simulation result with the following hyperparamters:

- $(n, n_{burn}, p) = (50000, 1000, 100)$
- Angle increment (in radians) $\theta = 0.05$, or, $\frac{2\pi}{\theta} \approx 126$
- $\text{Var}(z_t) = 10^{-2}$
- ε_t construction scale parameter is 10^{-2}
- x_t construction scale parameter is 10^{-1}

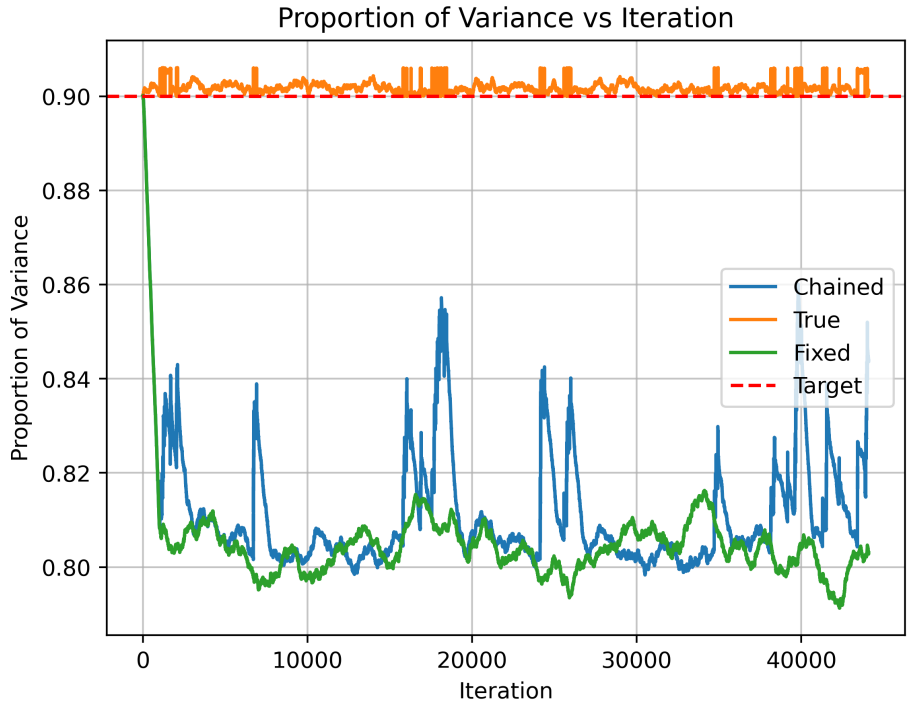


Figure 1: Proportion of variance retained by method.

We observe from Figure 1 that the Chained approach is generally more performant than with the fixed projection, however, it is not always better. Note that our projection chaining does not assume anything about the rotation structure; it just cares that the loadings are changing over time. One can entertain the idea that if a rotation structure is assumed, then all that is needed is to estimate the angle step in each plane, then in theory we circumvent the information loss issue. However, the rotation structure does not have an immediate real world application, so there is no grounds to even assume this when chaining projections.

Now

6.2 Static Kalman Filter

Simulations have been largely done, but still need to produce the plots. Will provide:

- Residual accuracy (based on a normalised version of $\text{var}(\tilde{y})$) using the estimated $\boldsymbol{\beta}$, maximised over initial choices of $P, Q \in \{10^{-4}I, 10^{-3}I, 10^{-2}I, 10^{-1}I\}$.
- Compare a few dimensions of β vs. $\hat{\beta}$.
- Time performance.

6.3 Dynamic Kalman Filter

Have only run small scale tests so far. Will produce same plots/tables as above.

7 Discussion

TO DO.

Bibliography

(https://math.stackexchange.com/users/29335/rschwieb), rschwieb (n.d.). *Define Rotation matrices for arbitrary dimension?* Mathematics Stack Exchange. URL:https://math.stackexchange.com/q/386513 (version: 2013-05-10). eprint: https://math.stackexchange.com/q/386513. URL: https://math.stackexchange.com/q/386513.

Li, Guo et al. (Jan. 2026). "High-dimensional Kalman filtering: From sparse modeling to AI-augmented architectures". en. In: *Signal Processing* 250, p. 110785. ISSN: 01651684. DOI: 10.1016/j.sigpro.2026.110785. URL: https://linkinghub.elsevier.com/retrieve/pii/S0165168426002987.